College of Computer Science and Information Technology
Department of Computer Engineering

ARTI 404 – Image Processing

Lab 2 Report: Sampling, Quantization, Arithmetic and Logical Operations

Student Name Mohammed Omar Halawani

Student ID 2240001174

Academic Year 2025 – 2026

1. Objective

This lab studies how sampling and quantization affect a digital image. It then applies arithmetic operations (addition, subtraction, adding a constant) and logical/set operations (union, intersection, difference, symmetric difference) to grayscale and binary images.

2. Images Used

- **lena_gray_256.tif** – grayscale image, used as image A.
- **cameraman.tif** – grayscale image, used as image B.
- **A.png** and **B.png** – binary images, each containing a white square, used for the union operation.

All images are loaded from the `images/` folder and resized to 400×400 where two images are combined.

3. Sampling and Quantization

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
def sample_image(image, factor):
    """
    Downsamples the image by the given factor.
    Args:
    image (numpy array): Original image.
    factor (int): Factor by which to downsample.
    Returns:
    numpy array: Downsampled image.
    """
    height, width = image.shape[:2]

    sampled_image = cv2.resize(image, (width // factor, height // factor),
interpolation=cv2.INTER_NEAREST)

    return sampled_image

def quantize_image(image, levels):
    """
    Reduces the number of grayscale levels in the image.
    Args:
    image (numpy array): Original image.
    levels (int): Number of grayscale levels.
    Returns:
    numpy array: Quantized image.
    """
    quantized_image = np.floor(image / (256 // levels)) * (256 // levels)
    quantized_image = quantized_image.astype(np.uint8)

    return quantized_image

def plot_images(original, sampled, quantized):
    """
    Plots the original, sampled, and quantized images side by side.
    Args:
    original (numpy array): Original image.
    sampled (numpy array): Sampled image.
    quantized (numpy array): Quantized image.
    """
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 3, 1)
    plt.imshow(original, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')
    plt.subplot(1, 3, 2)
    plt.imshow(sampled, cmap='gray')
    plt.title('Sampled Image')
    plt.axis('off')
    plt.subplot(1, 3, 3)
    plt.imshow(quantized, cmap='gray')
    plt.title('Quantized Image')
    plt.axis('off')
    plt.show()
```

```
image_path='images/lena_gray_256.tif'
sampling_factor=8
quantization_levels=32

# Load image in grayscale
original_image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if original_image is None:
    print(f"Error: Unable to load image at {image_path}")

# Sample and quantize
sampled_image = sample_image(original_image, sampling_factor)
quantized_image = quantize_image(original_image,
quantization_levels)

# Plot results
plot_images(original_image, sampled_image, quantized_image)
```

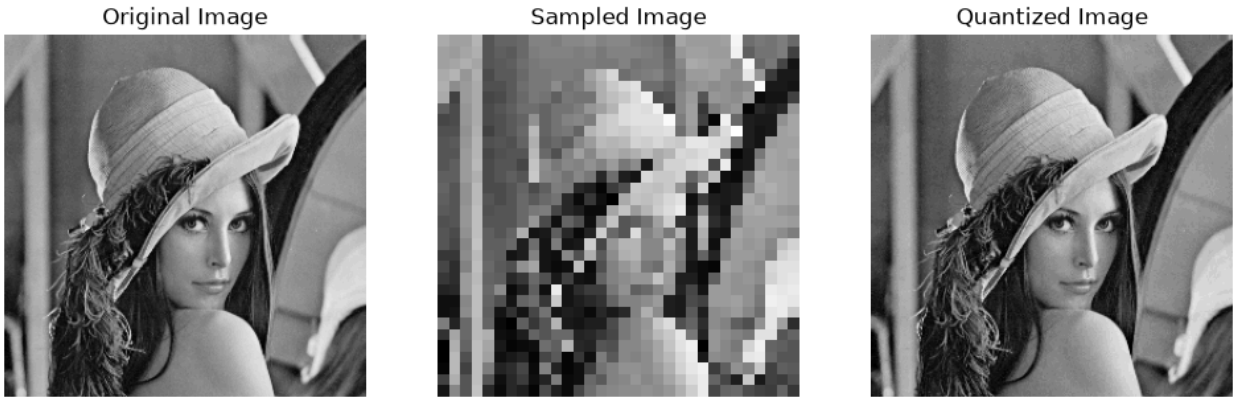


Figure 1: Original image, sampled with factor 8, and quantized to 32 gray levels.

Discussion: Downsampling by a factor of 8 with nearest-neighbour interpolation reduces the 256×256 image to 32×32 pixels. Most of the spatial detail is lost and the image becomes very blocky (pixelation). Quantizing to 32 levels makes each gray-level step 8 values wide. The result looks almost identical to the original: the eye cannot easily tell 32 levels from 256 in a natural image, although slight false contouring can appear in smooth areas. Reducing spatial resolution damages image quality far more than reducing intensity resolution does.

4. Arithmetic Operations

Addition

```
from PIL import Image

img1=Image.open('images/lena_gray_256.tif')
img2=Image.open ('images/cameraman.tif')
resize=(400,400)
img1=img1.resize(resize,Image.Resampling.LANCZOS)
img2=img2.resize(resize,Image.Resampling.LANCZOS)
im1arr=np.asarray(img1)
im2arr=np.asarray(img2)

addition=im1arr+im2arr

plt.imshow(addition, cmap='gray')
plt.title('Merged Image')
plt.axis('off')
plt.show()
```



Figure 2: Lena + cameraman using NumPy addition.

Discussion: Both arrays are `uint8`, so NumPy's + operator wraps around (modulo 256) when a sum goes over 255. The bright regions where both images are light therefore come out dark or black, which causes the sharp black patches and broken edges in the merged image. Saturating addition (`cv2.add`) or blending with `cv2.addWeighted` would avoid this.

Subtraction

```
subtraction = cv2.subtract(im1arr, im2arr)

plt.imshow(subtraction, cmap="gray")
plt.title("Image 1 - Image 2")
plt.axis("off")
plt.show()
```



Figure 3: Lena – cameraman using cv2.subtract.

Discussion: `cv2.subtract` saturates, so every pixel where the cameraman image is brighter than Lena, such as the sky, is clipped to 0 (black). Only the places where Lena is brighter remain visible, for example the hat and hair overlapping the dark coat and tripod.

Adding a constant

```
added_constant = cv2.add(im1arr, 175)

plt.imshow(added_constant, cmap="gray")
plt.title("Image + 175")
plt.axis("off")
plt.show()
```



Figure 4: Lena + 175 using cv2.add.

Discussion: The darkest Lena pixel is 23, so adding 175 shifts every pixel to 198 or above, and most pixels saturate at 255. The image becomes almost entirely white, and only faint outlines of the darkest regions (hair, eyes, background edges) remain. Matplotlib auto-scales this single plot, which exaggerates the remaining detail. In the combined figure below, with a fixed 0–255 display range, it looks almost blank.

5. Logical / Set Operations

Union (binary images)

```
img3=Image.open('images/A.png')
img4=Image.open('images/B.png')

resize=(400,400)
img3=img3.resize(resize,Image.Resampling.LANCZOS)
img4=img4.resize(resize,Image.Resampling.LANCZOS)

im3arr=np.asarray(img3)
im4arr=np.asarray(img4)

union=im4arr|im3arr
resultImage2=Image.fromarray(union)

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(img3, cmap='gray')
plt.title('Image A')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(img4, cmap='gray')
plt.title('Image B')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(resultImage2, cmap='gray')
plt.title('Union of A and B')
plt.axis('off')
plt.show()
```

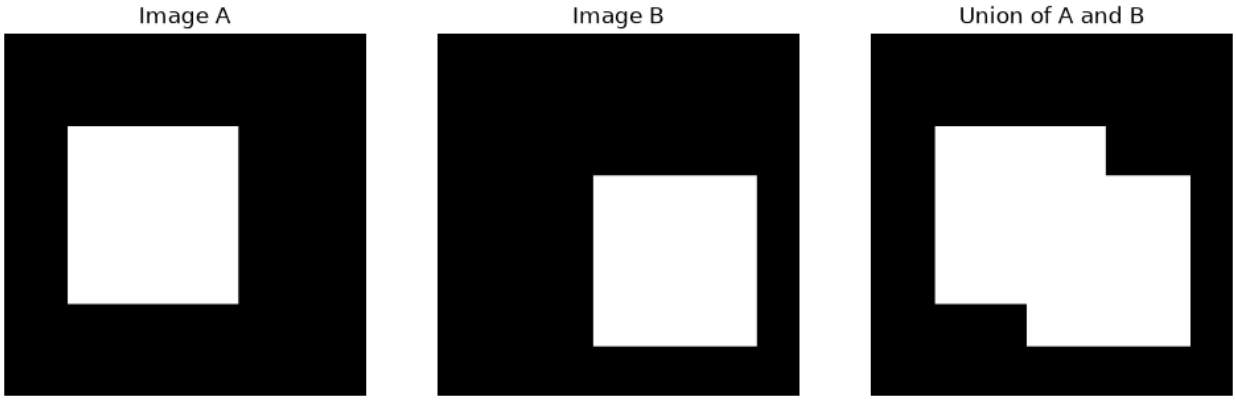


Figure 5: Binary images A and B and their union (A OR B).

Discussion: On binary images the bitwise OR gives a true set union. A pixel is white if it is white in A, in B or in both, so the result is the combined outline of the two overlapping squares.

Set difference, symmetric difference and intersection

These operations were applied to the grayscale images Lena (A) and cameraman (B):

```
difference = im1arr & (~im2arr)

plt.imshow(difference, cmap="gray")
plt.title("Set Difference: A - B")
plt.axis("off")
plt.show()
```

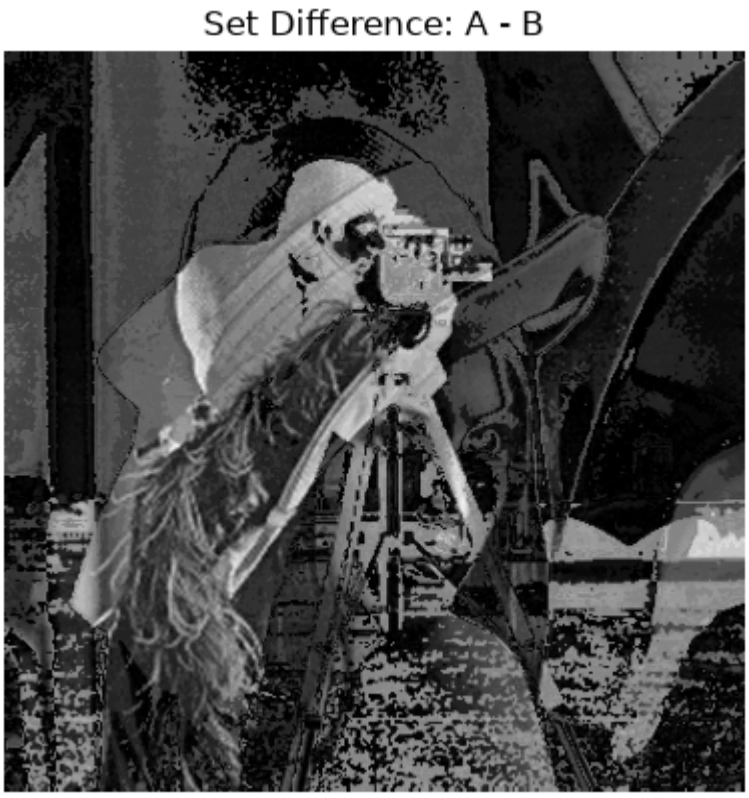


Figure 6: Set difference A AND (NOT B).

```
symmetric_difference = im1arr ^ im2arr

plt.imshow(symmetric_difference, cmap="gray")
plt.title("Symmetric Difference: A XOR B")
plt.axis("off")
plt.show()
```

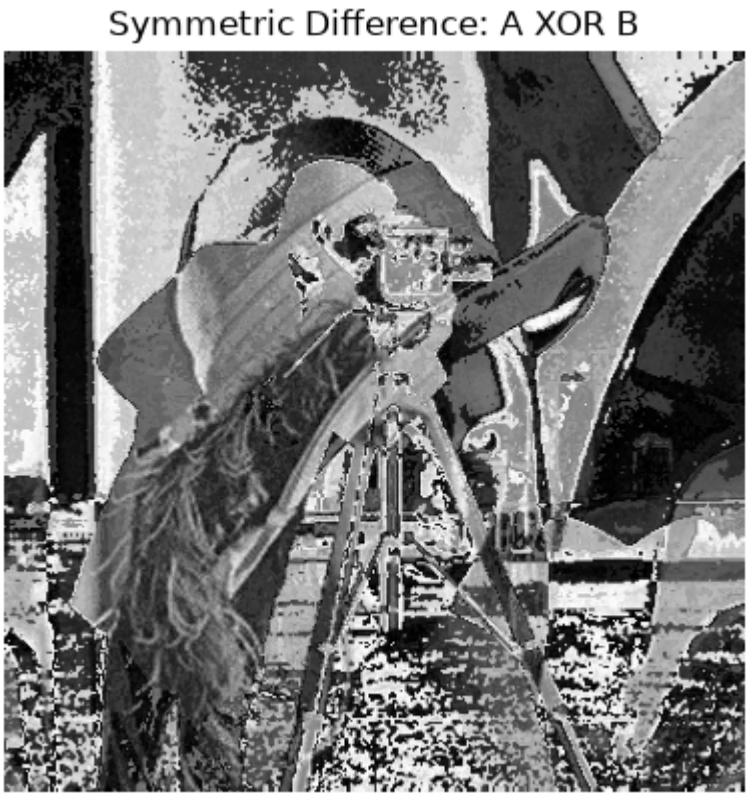


Figure 7: Symmetric difference A XOR B.

```
intersection = im1arr & im2arr

plt.imshow(intersection, cmap="gray", vmin=0, vmax=255)
plt.title("Intersection: A AND B")
plt.axis("off")
plt.show()
```

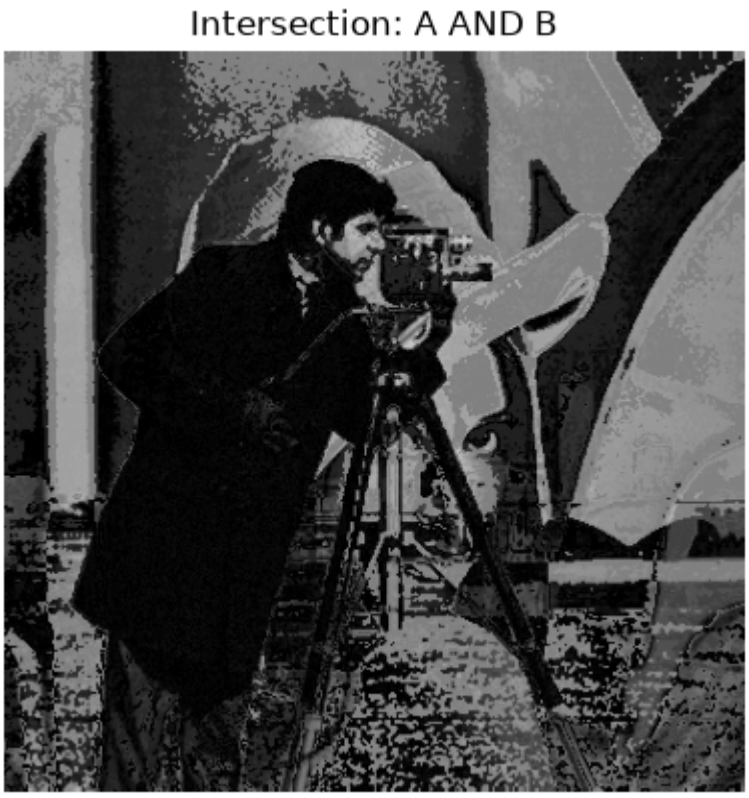


Figure 8: Intersection A AND B.

Discussion: On grayscale images these operators act on the individual bits of each 8-bit value, not on whole pixels. So the results are not clean set regions. They are bit-level mixes of the two images, which shows up as noisy, posterized textures. The intersection (AND) can only keep bits that are set in both images, so it is dominated by the darker of the two images, mainly the black coat and tripod. The XOR is brightest where the two images differ most. The set difference A AND (NOT B) keeps Lena's bits wherever the cameraman image is dark. These operators are best suited to binary masks, as in the union example.

6. Combined Results Using OpenCV

All operations were then repeated with OpenCV functions (`cv2.subtract`, `cv2.add`, `cv2.bitwise_and`, `cv2.bitwise_not`, `cv2.bitwise_xor`) and shown together with a fixed display range of 0–255:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load grayscale images
img1 = cv2.imread("images/lena_gray_256.tif", cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread("images/cameraman.tif", cv2.IMREAD_GRAYSCALE)

if img1 is None or img2 is None:
    raise FileNotFoundError("Unable to load one or both images.")

# Resize to same dimensions
img1 = cv2.resize(img1, (400, 400))
img2 = cv2.resize(img2, (400, 400))

# 1. Subtraction
subtraction = cv2.subtract(img1, img2)

# 2. Add constant 175
added_constant = cv2.add(img1, 175)

# 3. Set difference: A ∩ B'
set_difference = cv2.bitwise_and(
    img1,
    cv2.bitwise_not(img2)
)

# 4. Symmetric difference: A XOR B
symmetric_difference = cv2.bitwise_xor(img1, img2)

# 5. Intersection: A AND B
intersection = cv2.bitwise_and(img1, img2)


# Display results
images = [
    img1,
    img2,
    subtraction,
    added_constant,
    set_difference,
    symmetric_difference,
    intersection
]

titles = [
    "Image A",
    "Image B",
    "A - B",
    "A + 175",
    "Set Difference",
    "Symmetric Difference",
    "Intersection"
]

plt.figure(figsize=(14, 8))
```

```
for i, (image, title) in enumerate(zip(images, titles)):
    plt.subplot(2, 4, i + 1)
    plt.imshow(image, cmap="gray", vmin=0, vmax=255)
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Figure 9: Summary of all arithmetic and logical operations on Lena (A) and cameraman (B).

7. Conclusion

Spatial sampling has a much stronger visual effect than gray-level quantization. In arithmetic operations, the overflow behaviour matters: NumPy wraps values around, while OpenCV saturates them, and this gives very different results. Logical operations give meaningful set results on binary images, but on grayscale images they act bit by bit.